

SPA con React

4. Páginas (Router)



Universidad
Rey Juan Carlos

Micael Gallego

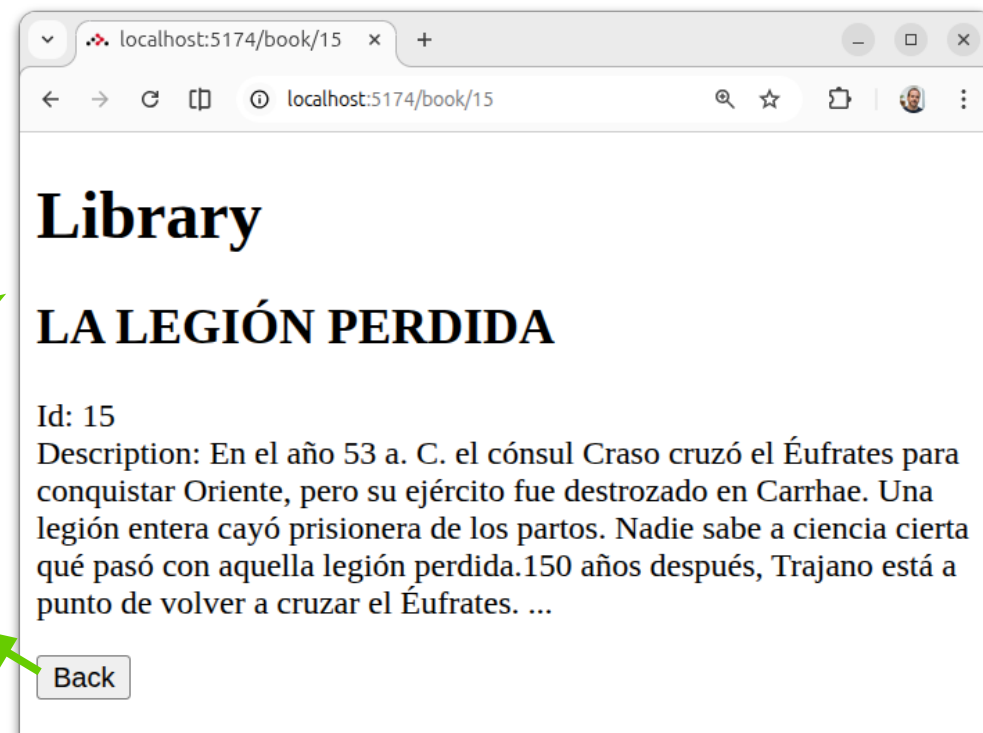
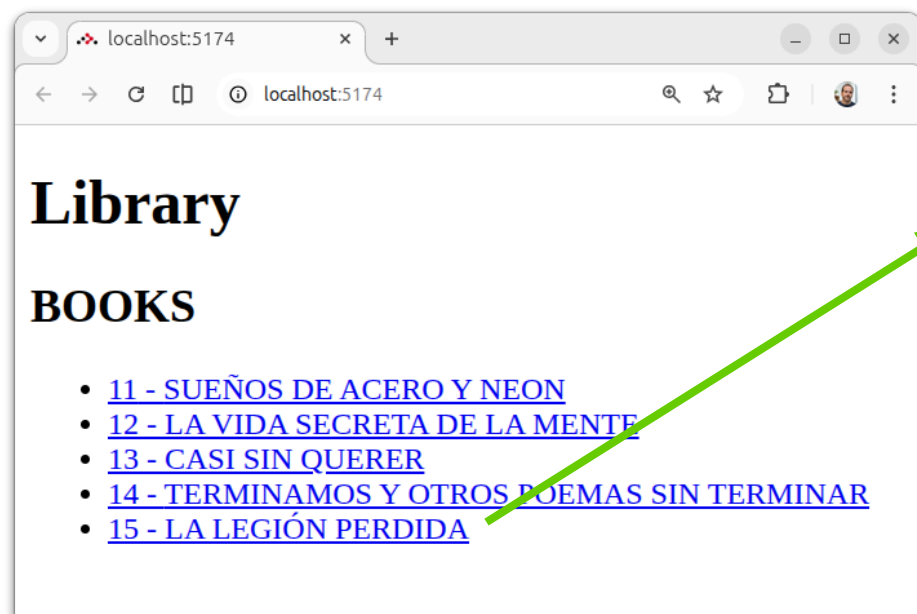
Correo: micael.gallego@urjc.es

Twitter: [@micael_gallego](https://twitter.com/micael_gallego)



Páginas (Router)

- Las **webs SPA** (*single page application*) pueden simular la navegación en **varias páginas** cambiando de forma dinámica la URL (**ruta**) y el componente que se muestra



Páginas (Router)

- La librería **React** por defecto no permite simular la navegación
- Es necesario una librería de **enrutamiento** (*routing*)
- Los **frameworks React** gestionan el enrutamiento de formas muy diferentes (usamos **React Router**)



NEXT.js



Páginas (Router)

- El **componente principal** de la aplicación (**home.tsx**) puede tener una parte fija (**cabecera, footer**) y una parte cuyo **contenido** depende de la **URL** (**<Outlet/>**)
- En **app/routes.ts** se define qué **componente** se muestra en el **outlet** para cada **URL**
- Existen **links especiales** para navegar dentro de la aplicación web (**<Link>...</Link>**)
- Desde el código se puede navegar de forma programática (**useNavigate()**)

<https://reactrouter.com/start/framework/routing>

Páginas (Router)

- Dos páginas (BookList y BookDetail) usando un store (BookStore)

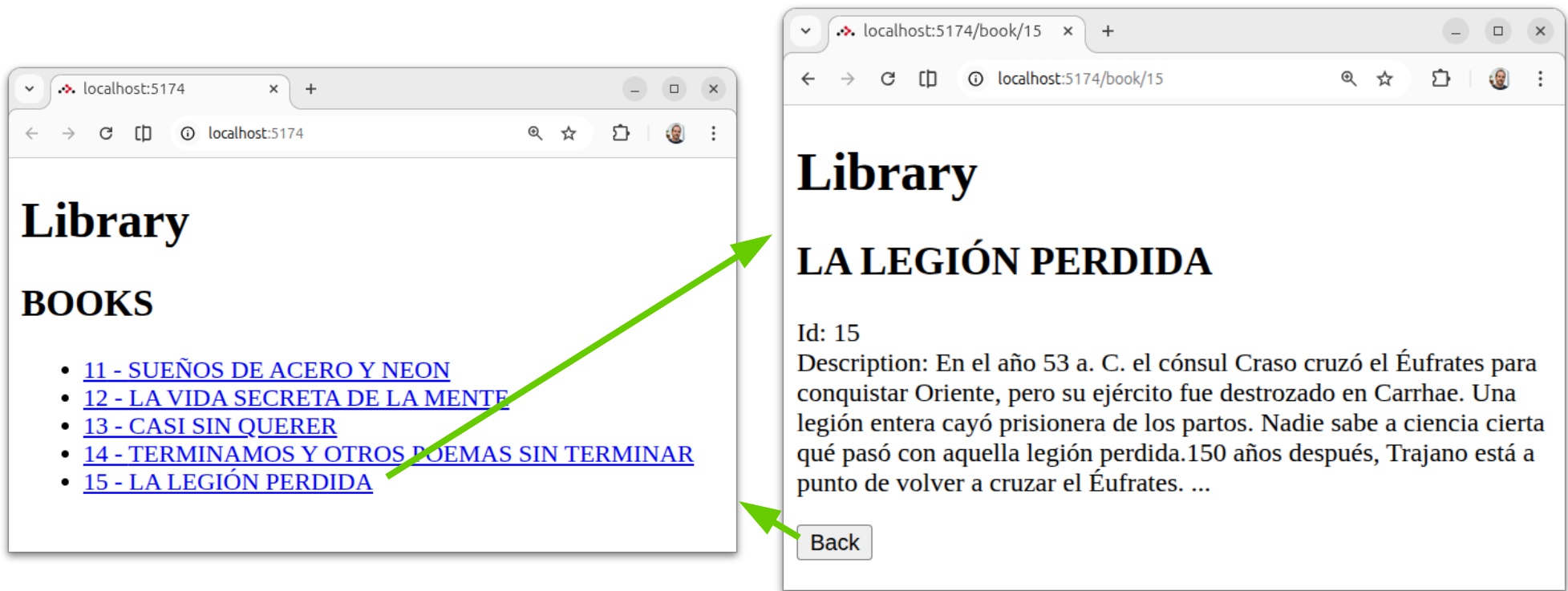
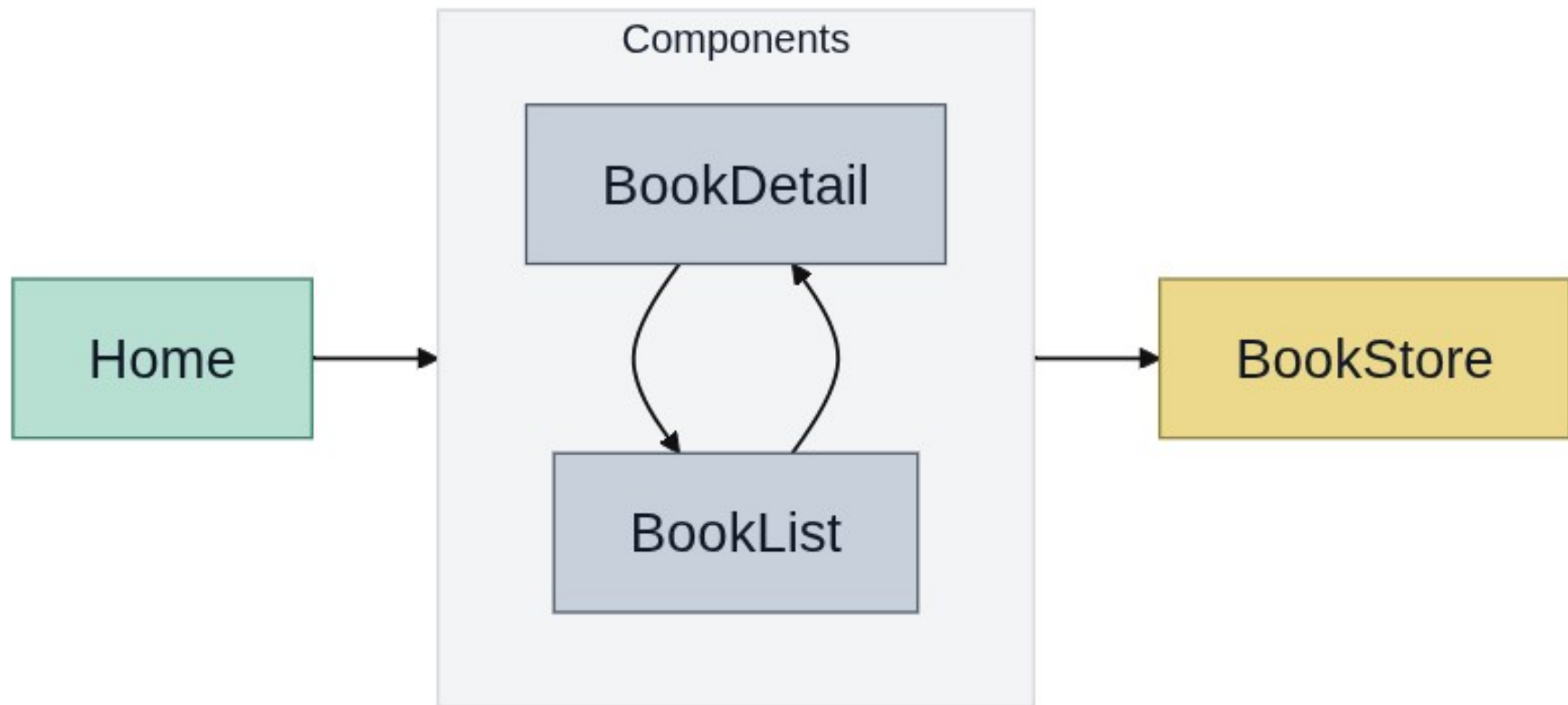


Diagrama de elementos



Componente principal

home.tsx

```
import { Outlet } from "react-router";  
  
export default function Home() {  
  return (  
    <div>  
      <h1>Library</h1>  
      <Outlet />  
    </div>  
  );  
}
```

Componente principal

home.tsx

```
import { Outlet } from "react-router";  
  
export default function Home() {  
  return (  
    <div>  
      <h1>Library</h1>  
      <Outlet />  
    </div>  
  );  
}
```

Zona que cambia en
función de la URL

Configuración de las rutas

routes.ts

```
import { type RouteConfig, index, layout, route }
  from "@react-router/dev/routes";

export default [
  layout("routes/home.tsx", [
    index("routes/book-list.tsx"),
    route("book/:id", "routes/book-detail.tsx"),
  ]),
] satisfies RouteConfig;
```

Configuración de las rutas

routes.ts

```
import { type RouteConfig, index, layout
  from "@react-router/dev/routes";

export default [
  layout("routes/home.tsx", [
    index("routes/book-list.tsx"),
    route("book/:id", "routes/book-detail.tsx"),
  ]),
] satisfies RouteConfig;
```

Componente que define el diseño
de las pantallas (*layout*)
Parte fija y parte variable

Componente en la raíz

Componente de la URL book/:id

Componente BookList

book-list.tsx

```
import { Link } from "react-router";
import { useBooksStore } from "~/stores/books-store";

export default function BookList() {
  const { books } = useBooksStore();

  return (
    <div>
      <h2>BOOKS</h2>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <Link to={` /book/${book.id}`}>
              {book.id} - {book.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Componente BookList

book-list.tsx

```
import { Link } from "react-router";
import { useBooksStore } from "~/stores/books-store";

export default function BookList() {
  const { books } = useBooksStore();

  return (
    <div>
      <h2>BOOKS</h2>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <Link to={`/book/${book.id}`}>
              {book.id} - {book.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

En vez de <a>, los links a otras páginas son <Link> y la URL se indicar con **to**

Páginas (Router)

exam31_routes

Componente BookDetail

book-detail.tsx

```
import { useNavigate, useParams } from "react-router";
import { useBooksStore } from "~/stores/books-store";

export default function BookDetail() {

  const { id } = useParams();
  const navigate = useNavigate();
  const { getBook } = useBooksStore();

  const book = getBook(id!);

  if (!book) { return <p>Book not found</p>; }

  return (
    <div>
      <h2>{book.title}</h2>
      <div><label>Id: </label>{book.id}</div>
      <div><label>Description: </label>{book.description}</div>
      <p><button onClick={() => navigate("/")}>Back</button></p>
    </div>
  );
}
```

Componente BookDetail

book-detail.tsx

```
import { useNavigate, useParams } from "react-router";
import { useBooksStore } from "~/stores/books-store";

export default function BookDetail() {

  const { id } = useParams();
  const navigate = useNavigate();
  const { getBook } = useBooksStore();

  const book = getBook(id!);

  if (!book) { return <p>Book not found</p>; }

  return (
    <div>
      <h2>{book.title}</h2>
      <div><label>Id: </label>{book.id}</div>
      <div><label>Description: </label>{book.description}</div>
      <p><button onClick={() => navigate("/")}>Back</button></p>
    </div>
  );
}
```

Para acceder a los parámetros de la URL se usa **useParams()**

Páginas (Router)

exam31_routes

Componente BookDetail

book-detail.tsx

```
import { useNavigate, useParams } from "react-router";
import { useBooksStore } from "~/stores/books-store";

export default function BookDetail() {

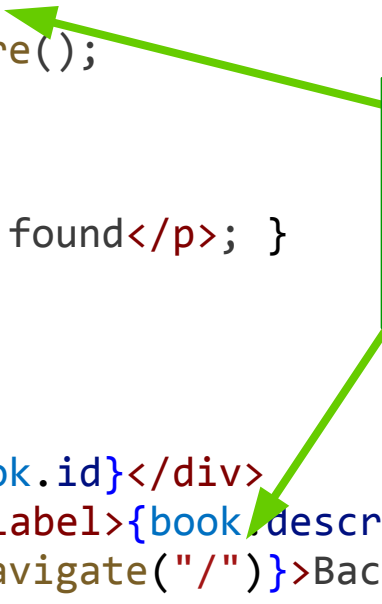
  const { id } = useParams();
  const navigate = useNavigate();
  const { getBook } = useBooksStore();

  const book = getBook(id!);

  if (!book) { return <p>Book not found</p>; }

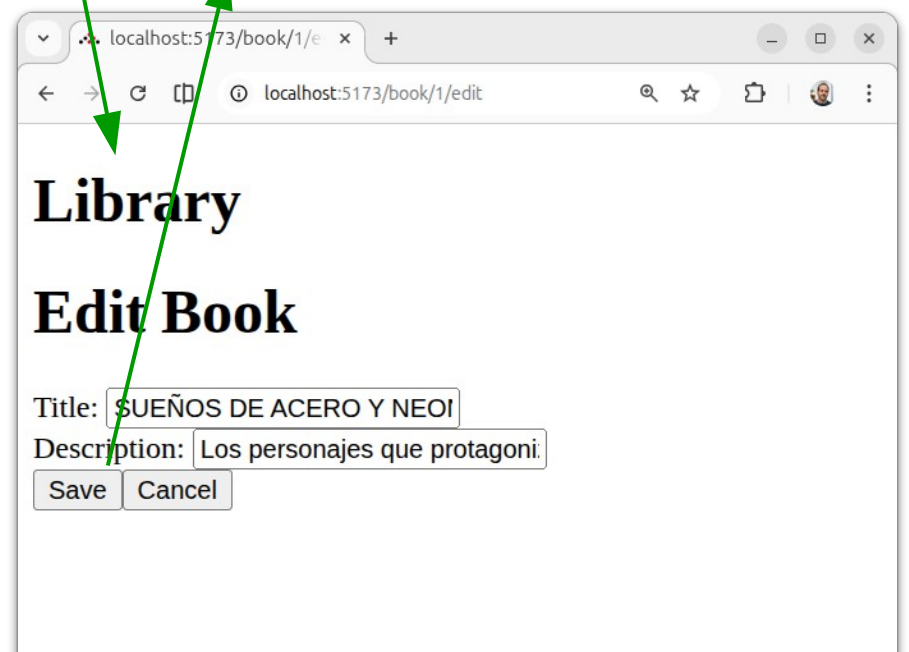
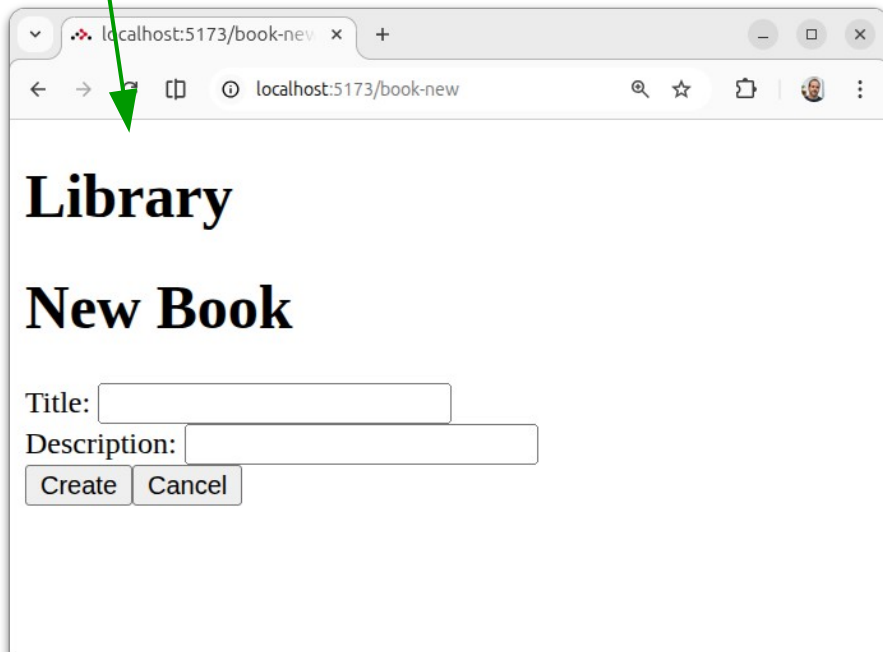
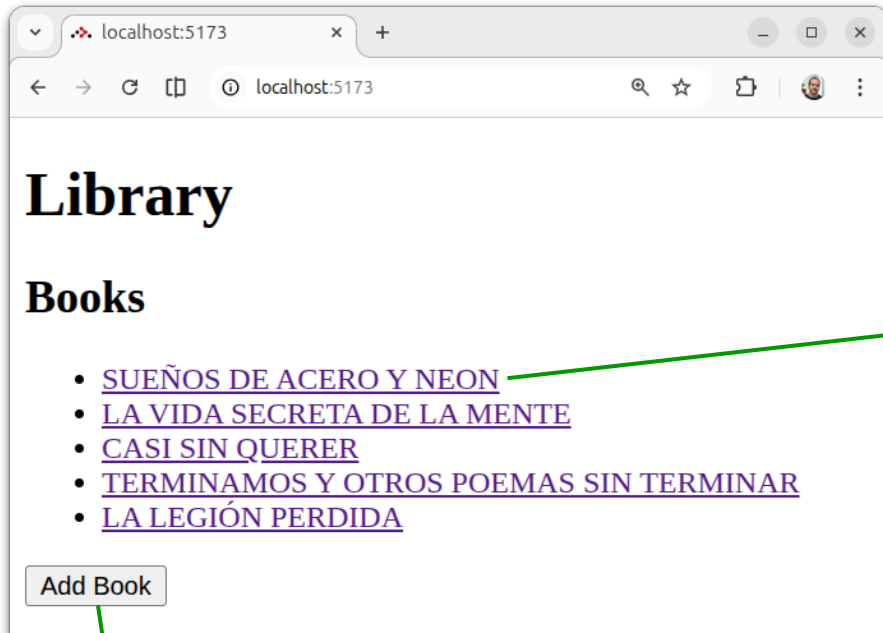
  return (
    <div>
      <h2>{book.title}</h2>
      <div><label>Id: </label>{book.id}</div>
      <div><label>Description: </label>{book.description}</div>
      <p><button onClick={() => navigate("/")}>Back</button></p>
    </div>
  );
}
```

Para navegar se usar
navigate(...) de
useNavigate()



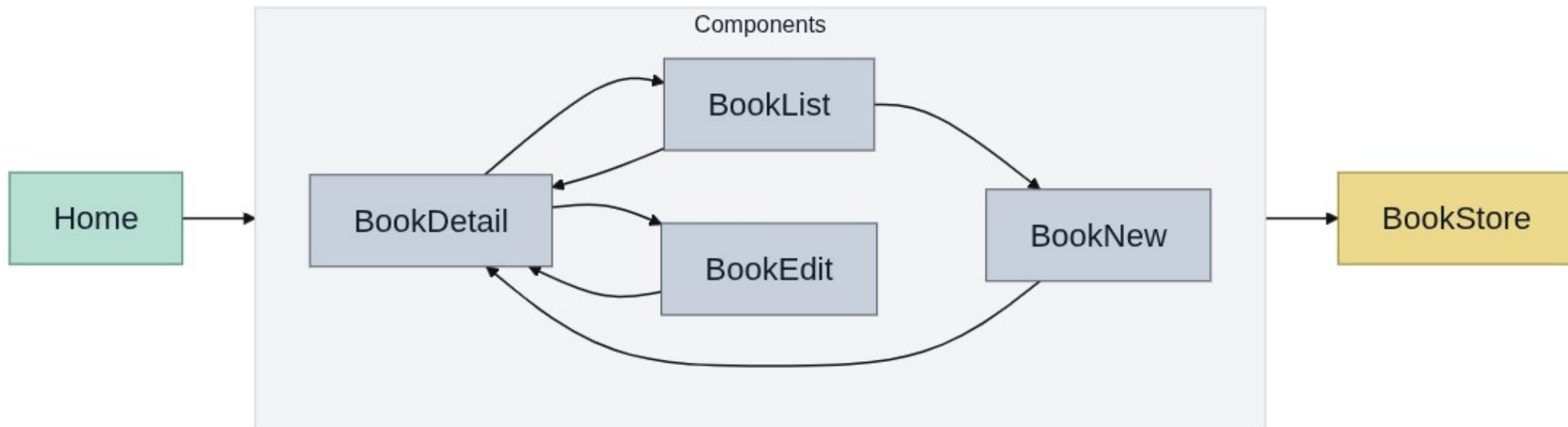
Ejercicio 8

- Amplía el ejemplo anterior para incluir la creación, modificación y borrado de libros:
 - Cambios en el store
 - Nuevo libro
 - Borrar libro
 - Actualizar libro
 - Nuevas páginas
 - Nuevo libro
 - Modificación de libro



Ejercicio 8

Diagrama de elementos



Gestión del estado con API REST

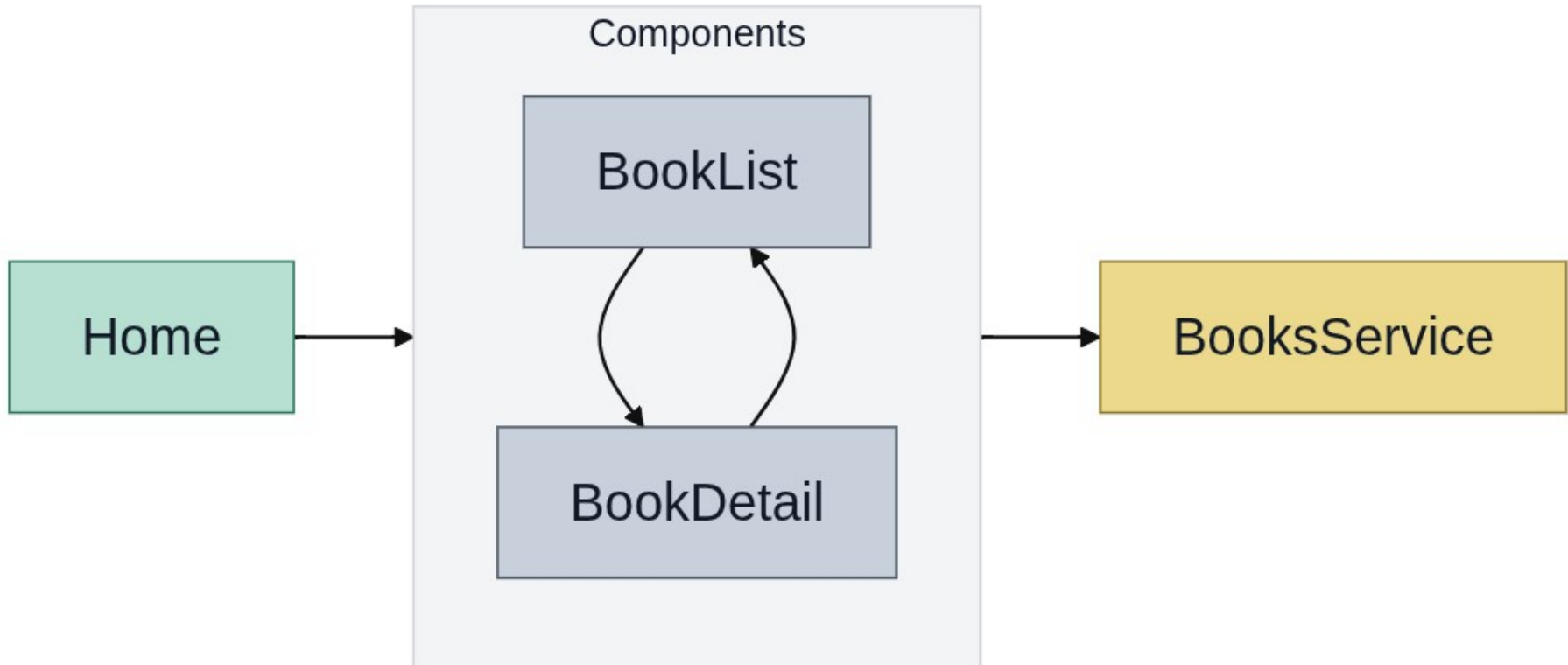
- En aplicaciones multipágina con API REST hay que decidir cómo gestionar el estado
- **Datos en memoria (con estado):**
 - Se mantiene en memoria los datos cargados de la API REST
 - Ventaja: Navegación más fluida
 - Desventaja: Diseñar estrategia para actualizar los datos de memoria (cuándo?)
- **Datos se cargan al navegar entre páginas (sin estado):**
 - En cada pantalla se cargan los datos de la API REST
 - Ventaja: Implementación más sencilla. Datos siempre actualizados
 - Desventaja: En cada transición de pantalla se usa API REST (lento)
 - **Opción elegida aquí por simplicidad**

Gestión del estado con API REST

- Para implementar una app React sin estado **no se necesita un store** (no hay estado compartido)
- En cada pantalla se **carga la información** necesaria desde la API REST (**useEffect**)
- Los formularios de creación y modificación se procesan con **useActionState**
- Las operaciones de la API REST se implementan en **un servicio** (para que el código sea más mantenible)

Gestión del estado con API REST

Diagrama de elementos



Gestión del estado con API REST

books-service.ts

```
import type { Book } from "~/models/book";

export async function getBooks(): Promise<Book[]> {
  const response = await fetch('https://openlibrary.org/search.json?title=react&fields=title,key&limit=10');

  if (!response.ok) {
    throw new Error(`Error fetching books: ${response.statusText}`);
  }
  const data = await response.json();

  if (data.docs) {
    return data.docs.map((item: any) => ({
      id: item.key.split("/").pop(),
      title: item.title
    }));
  }

  return [];
}

export async function getBook(id: string): Promise<Book> {
  const response = await fetch(`https://openlibrary.org/works/${id}.json`);
  if (!response.ok) {
    throw new Error(`Error fetching book: ${response.statusText}`);
  }
  const data = await response.json();

  const description = typeof data.description === 'string'
    ? data.description
    : data.description?.value || "No description";

  return { id, title: data.title, description };
}
```

Gestión del estado con API REST

book-list.tsx

```
export default function BookList() {
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function loadBooks() {
    setLoading(true);
    setError(null);
    try {
      setBooks(await getBooks());
    } catch (err) {
      console.error(err);
      setError(err instanceof Error ? err.message : "Error loading books");
      setBooks([]);
    } finally {
      setLoading(false);
    }
  }

  useEffect (() => { loadBooks() }, []);

  return (
    <div>
      <h2>BOOKS</h2>
      {loading && <p>Loading...</p>}
      {error && <p style={{ color: "red" }}>{error}</p>}
      <ul>
        {books.map((book) => (
          <li key={book.id}><Link to={` /book/${book.id}`}>{book.id} - {book.title}</Link></li>
        ))}
      </ul>
    </div>
  );
}
```

Se cargan los datos
usando el servicio de
libros

Gestión del estado con API REST

book-detail.tsx

```
export default function BookDetail() {

  const { id } = useParams();
  const navigate = useNavigate();

  const [book, setBook] = useState<Book>();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function loadBook() {
    setLoading(true);
    setError(null);
    try {
      setBook(await getBook(id!));
    } catch (err) {
      console.error(err);
      setError(err instanceof Error ? err.message : "Error loading book");
      setBook(undefined);
    } finally {
      setLoading(false);
    }
  }

  useEffect (() => { loadBook() },[id]);

  if(loading) { return <p>Loading...</p>; }
  if (error) { return <p style={{ color: "red" }}>{error}</p>; }
  if (!book) { return <p>Book not found</p>; }

  return (
    <div> ... </div>
  );
}
```

Gestión del estado con API REST

book-detail.tsx

```
export default function BookDetail() {

  const { id } = useParams();
  const navigate = useNavigate();

  const [book, setBook] = useState<Book>();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function loadBook() {
    setLoading(true);
    setError(null);
    try {
      setBook(await getBook(id!));
    } catch (err) {
      console.error(err);
      setError(err instanceof Error ? err.message : "Error loading book");
      setBook(undefined);
    } finally {
      setLoading(false);
    }
  }

  useEffect (() => { loadBook() },[id]);

  if(loading) { return <p>Loading...</p>; }
  if (error) { return <p style={{ color: "red" }}>{error}</p>; }
  if (!book) { return <p>Book not found</p>; }

  return (
    <div> ... </div>
  );
}
```

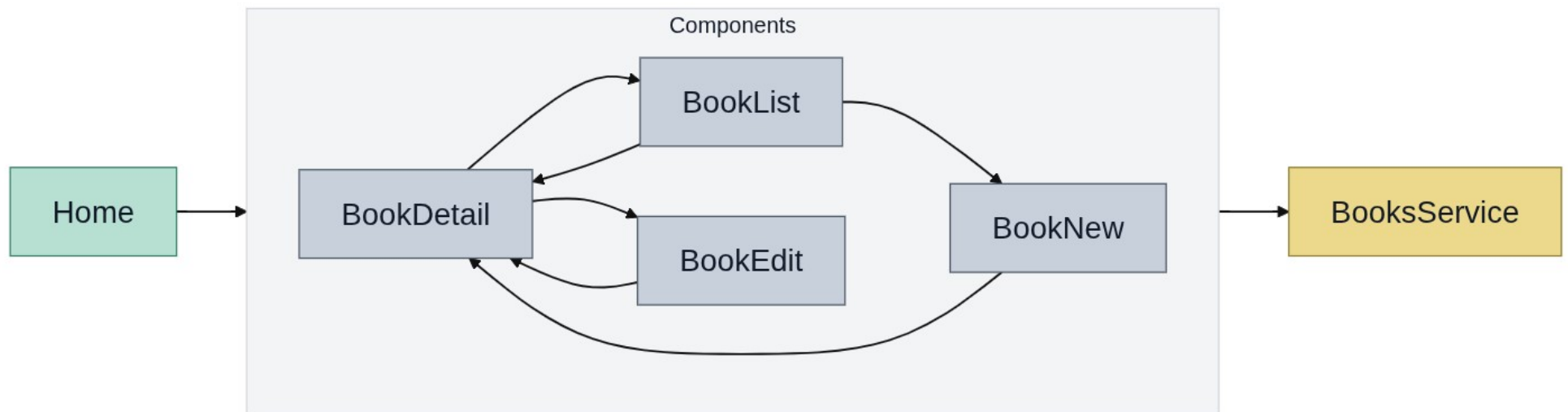
Se cargan los datos
usando el servicio de
libros

Ejercicio 9

- Actualiza el ejercicio 8 (libros) para que los datos se gestionen en el backend
- La comunicación se realizará con API REST
- Se seguirá el enfoque sin estado
 - Los datos se cargan del servidor al cargar cada pantalla
 - El store se transforma en un servicio que hace llamadas a la API REST

Ejercicio 9

Diagrama de elementos



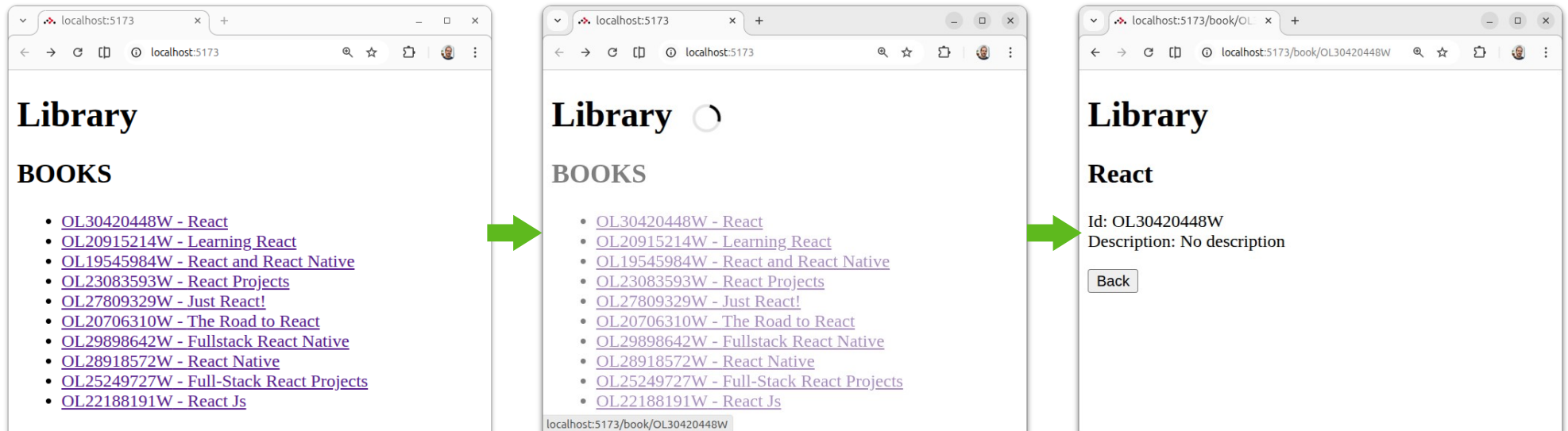
Ejercicio 9

- Se proporciona una **API REST** (similar a la de los items pero para books).
- Cada libro tiene las propiedades: id, title, description
- Ejecución
 - Java
 - Requisitos: Java & Maven
 - Comando: mvn spring-boot:run
 - Node
 - Requisitos: Node.js y NPM
 - Comando: npm install & npm start

Carga antes de mostrar la página

- La mayoría de las pantallas tienen **código repetitivo** para gestionar la carga de datos y los errores
- El código repetitivo que **dificulta la comprensión** de la lógica de negocio
- React Router permite simplificar este patrón con los **loaders**
- Cada componente define una función **clientLoader** para cargar los datos (disponibles con **useLoaderData(...)**)
- La visualización de la **carga** y los **errores** se pueden gestionar **globalmente**

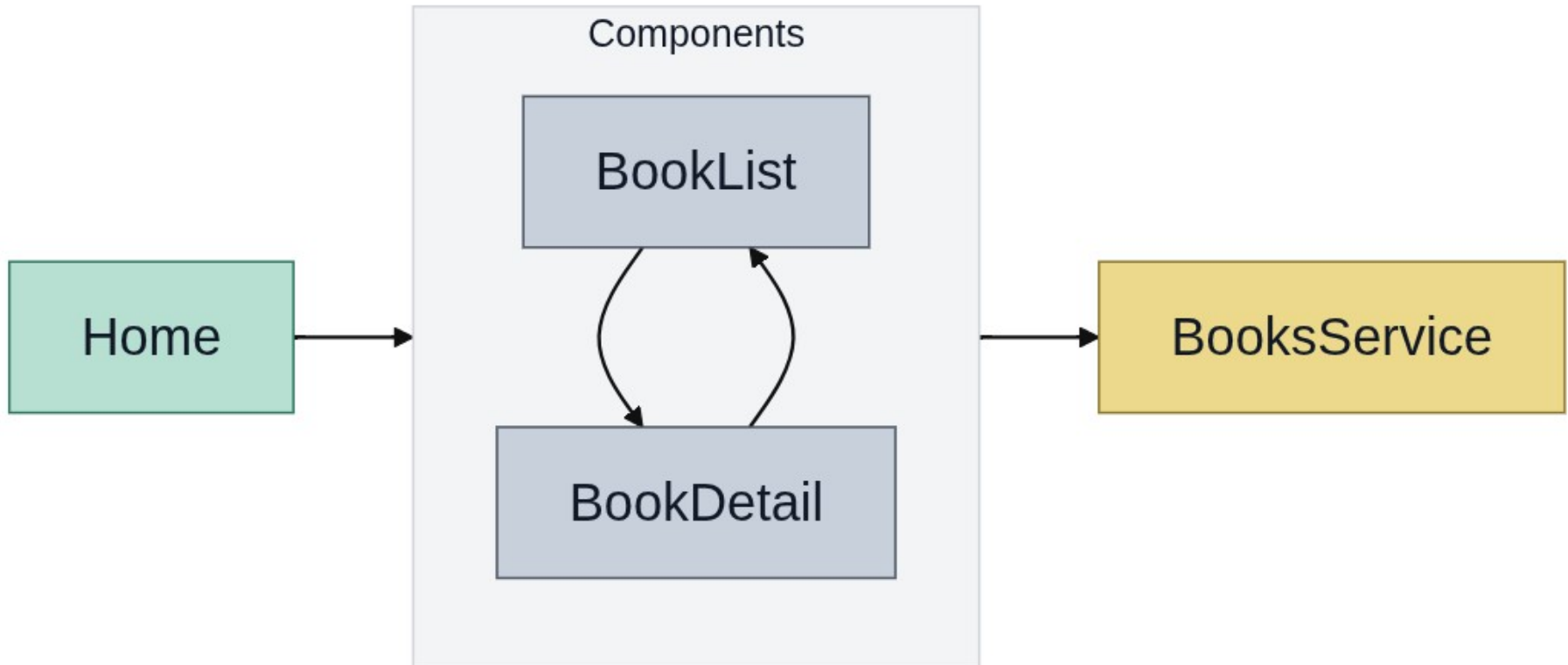
Carga antes de mostrar la página



Carga antes de mostrar la página

exam33_routes_loader

Diagrama de elementos



Carga antes de mostrar la página

```
import { Link, useLoaderData } from "react-router";
import { getBooks } from "~/services/books-service";
import type { Route } from "../types/book-list";

export async function clientLoader() {
  return await getBooks();
}

export default function BookList({ loaderData }: Route.ComponentProps) {

  const books = loaderData;

  return (
    <div>
      <h2>BOOKS</h2>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <Link to={` /book/${book.id}`}>
              {book.id} - {book.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Carga antes de mostrar la página

```
import { Link, useLoaderData } from "react-router";
import { getBooks } from "~/services/books-service";
import type { Route } from "../types/book-list";

export async function clientLoader() {
  return await getBooks();
}

export default function BookList({ loaderData }: Route.ComponentProps) {

  const books = loaderData;

  return (
    <div>
      <h2>BOOKS</h2>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <Link to={`/book/${book.id}`}>
              {book.id} - {book.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Función de carga

Datos cargados

Carga antes de mostrar la página

```
import { useNavigate, useLoaderData } from "react-router";
import { getBook } from "~/services/books-service";
import type { Route } from "../types/book-detail";

export async function clientLoader({ params }: Route.ClientLoaderArgs) {
  return await getBook(params.id!);
}

export default function BookDetail({ loaderData }: Route.ComponentProps) {

  const book = loaderData;

  const navigate = useNavigate();

  return (
    <div>
      <h2>{book.title}</h2>
      <div>
        <label>Id: </label>{book.id}
      </div>
      <div>
        <label>Description: </label>{book.description}
      </div>
      <p>
        <button onClick={() => navigate("/")}>Back</button>
      </p>
    </div>
  );
}
```

Carga antes de mostrar la página

```
import { useNavigate, useLoaderData } from "react-router";
import { getBook } from "~/services/books-service";
import type { Route } from "../types/book-detail";

export async function clientLoader({ params }: Route.ClientLoaderArgs) {
  return await getBook(params.id!);
}

export default function BookDetail({ loaderData }: Route.ComponentProps) {

  const book = loaderData;

  const navigate = useNavigate();

  return (
    <div>
      <h2>{book.title}</h2>
      <div>
        <label>Id: </label>{book.id}
      </div>
      <div>
        <label>Description: </label>{book.description}
      </div>
      <p>
        <button onClick={() => navigate("/")}>Back</button>
      </p>
    </div>
  );
}
```

Función de carga

Datos cargados

Carga antes de mostrar la página

```
import { Outlet, useNavigation, isRouteErrorResponse } from "react-router";
import type { Route } from "../types/home";
import "../home.css";

export default function Home() {

  const navigation = useNavigation();

  const isLoading = navigation.state === "loading";

  return (
    <div>
      <h1>Library {isLoading && <span className="spinner"></span>}</h1>
      <div style={{ opacity: isLoading ? 0.5 : 1 }}>
        <Outlet />
      </div>
    </div>
  );
}

export function ErrorBoundary({ error }: Route.ErrorBoundaryProps) {
  let errorMessage = error instanceof Error ? error.message : "An unexpected error occurred";
  return (
    <div>
      <h1>Library</h1>
      <div style={{ backgroundColor: "lightcoral" }}>
        <p>Error: {errorMessage}</p>
        <button onClick={() => window.location.href = "/"}>Back to home</button>
      </div>
    </div>
  );
}
```

Carga antes de mostrar la página

```
import { Outlet, useNavigation, isRouteErrorResponse } from "react-router";
import type { Route } from "../types/home";
import "../home.css";

export default function Home() {
  const navigation = useNavigation();
  const isLoading = navigation.state === "loading";

  return (
    <div>
      <h1>Library {isLoading && <span className="spinner"></span>}</h1>
      <div style={{ opacity: isLoading ? 0.5 : 1 }}>
        <Outlet />
      </div>
    </div>
  );
}

export function ErrorBoundary({ error }: Route.ErrorBoundaryProps) {
  let errorMessage = error instanceof Error ? error.message : "An unexpected error occurred";
  return (
    <div>
      <h1>Library</h1>
      <div style={{ backgroundColor: "lightcoral" }}>
        <p>Error: {errorMessage}</p>
        <button onClick={() => window.location.href = "/"}>Back to home</button>
      </div>
    </div>
  );
}
```

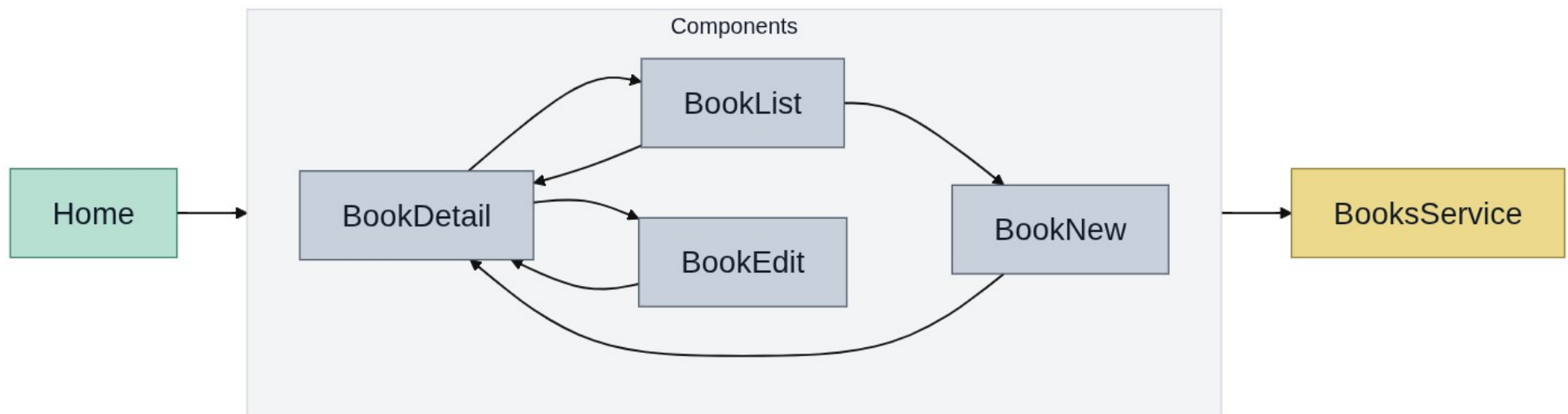
El interfaz cambia
cuando se está cargando

Y cuando falla la
carga

Ejercicio 10

- Refactoriza el ejercicio 9 para que use los loaders de React Router

Diagrama de elementos



- **Funcionalidades avanzadas**

- Rutas hijas (child routes) que se aplican a un componente concreto (en vez de las aplicadas a toda la app)
- Ejecutar código al salir de una pantalla
 - Si el usuario navega a otra página “sin guardar” se le puede preguntar si realmente desea descartar los cambios o abortar la navegación
- Verificar si se puede ir a una nueva pantalla
 - Generalmente se comprueba si el usuario tiene permisos para hacerlo
- Las pantallas sólo se cargan al acceder a ellas (*lazy loading*)
- Animaciones en las transiciones entre pantallas

Páginas (Router)

- Se puede **simular la navegación** en una aplicación SPA (aunque haya sólo una página real)
- Los **frameworks** siguen estrategias muy diferentes (React Router, Next.js, Tanstack...)
- Se pueden mantener **partes fijas** y **partes diferentes** en las páginas (header, footer)
- Aplicaciones **sin estado** son más **sencillas** de implementar pero más **lentas** (no necesitan *store*)
- El router puede **cargar** los datos **antes de mostrar** la página (código sencillo y cargador global)